REPORTE DE PRÁCTICAS: EJERCICIOS PARES (PILAS)

Ángel de Jesús Nolasco Rivera

Grupo: GITID244

Fecha: 03/11/2026

Materia: Estructura de Datos

Ejercicio 2: 4086. Conjunto

1. DESCRIPCIÓN BREVE DEL PROBLEMA

El problema consiste en determinar si una cadena de texto dada pertenece al conjunto S. Dicho conjunto permite exclusivamente los caracteres 'a', 'b', 'c', 'd' y paréntesis '(' y ')' asegurando que estos últimos estén perfectamente balanceados en su aperturas y cierres.

2. EXPLICACIÓN DEL ALGORITMO

Se realiza un recorrido carácter por carácter manteniendo un contador de balanceo. Se ignoran o aceptan directamente las letras 'a'-'d'. Al encontrar un '(' se incrementa el contador y con ')' se decrementa. Si el contador resulta menor a cero en algún punto o diferente de cero al finalizar, la cadena no es válida. La solución modular utiliza dos archivos: ValidadorConjunto.java para la regla y Main.java para la entrada/salida.

3. PRUEBAS REALIZADAS Y VEREDICTO (OMEGAUP)

Fecha y hora	Lenguaje	Porcentaje	Ejecución	Salida	Memoria	Tiempo	Acciones
2026-11-03 22:30	java	100.00%	Terminada	Correcta	1.27 MB	0.61 s	

Puntaje obtenido: 12.34 / 12.34 pts (100% Correcto)

4. ENLACE AL CÓDIGO EN GITHUB

Repositorio: https://github.com/ANGEL7231/angel-nolasco-pilas-java/tree/main/ejercicio1_Conjunto

5. CONCLUSIONES Y DIFICULTADES

Dificultad: Manejar adecuadamente los casos donde aparecen caracteres fuera del rango permitido o con cierres prematuros.
Solución: Se implementó una verificación inmediata devolviendo falso si el contador cae por debajo de cero durante el flujo del ciclo.

Ejercicio 4: 5308. ()[]{}<>

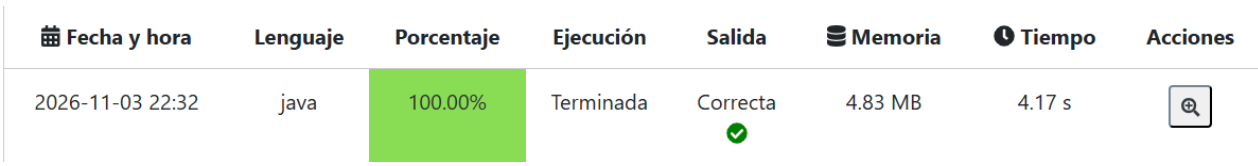
1. DESCRIPCIÓN BREVE DEL PROBLEMA

Dada una cadena de símbolos de agrupación ('(', ')', '[', ']', '{', '}', '<', '>'), se debe calcular la menor cantidad de sustituciones de símbolos de apertura por otros de apertura (o cierre por cierre) para lograr que la cadena esté balanceada. Si no es posible balancear, se imprime "imposible".

2. EXPLICACIÓN DEL ALGORITMO

Se utiliza una pila de caracteres (Stack<Character>). Al encontrar un símbolo de apertura se inserta en la pila. Al leer un símbolo de cierre, se desapila el último elemento; si el tipo no coincide con el de cierre, se incrementa un contador de reemplazos. Si la longitud es impar o la pila queda desbalanceada al final, es imposible.

3. PRUEBAS REALIZADAS Y VEREDICTO (OMEGAUP) - EJERCICIO 4



Fecha y hora	Lenguaje	Porcentaje	Ejecución	Salida	Memoria	Tiempo	Acciones
2026-11-03 22:32	java	100.00%	Terminada	Correcta	4.83 MB	4.17 s	

4. ENLACE AL CÓDIGO EN GITHUB

Repositorio: https://github.com/ANGEL7231/angel-nolasco-pilas-java/tree/main/ejercicio2_Balanceo

5. CONCLUSIONES Y DIFICULTADES

Dificultad: Diferenciar entre un desbalance estructural insalvable y una simple sustitución de tipo de paréntesis.
Solución: Se usó la pila de apertura estricta y se descartó inmediatamente cualquier cadena con longitud impar o con exceso de cierres.

Ejercicio 6: 6818. Los Últimos Serán Los Primeros

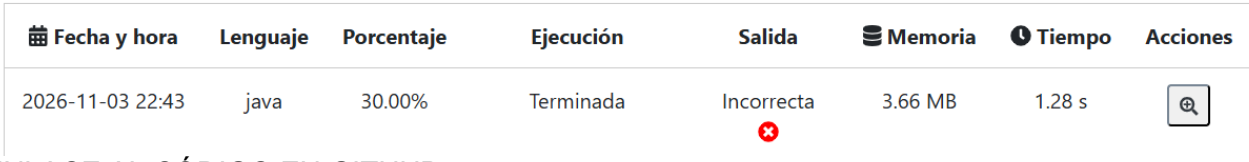
1. DESCRIPCIÓN BREVE DEL PROBLEMA

En una carrera especial en Yahualica, los competidores ganan en orden inverso a su llegada a la meta. Se recibe una secuencia de nombres finalizada con el carácter '#' y se deben imprimir los nombres empezando por el último en cruzar la meta.

2. EXPLICACIÓN DEL ALGORITMO

El problema ejemplifica de forma directa el comportamiento LIFO (Last In, First Out). Se leen los nombres en orden de entrada y se insertan (push) en una pila de cadenas. Al leer '#', se detiene la

entrada y se vacía la pila imprimiendo cada elemento mediante pop().
3. PRUEBAS REALIZADAS Y VEREDICTO (OMEGAUP)



Fecha y hora	Lenguaje	Porcentaje	Ejecución	Salida	Memoria	Tiempo	Acciones
2026-11-03 22:43	java	30.00%	Terminada	Incorrecta	3.66 MB	1.28 s	

4. ENLACE AL CÓDIGO EN GITHUB

Repositorio: https://github.com/ANGEL7231/angel-nolasco-pilas-java/tree/main/ejercicio3_ultimos_primeros

5. CONCLUSIONES Y DIFICULTADES

Dificultad: Controlar el centinela '#' sin incluirlo dentro de la lista de salida.
Solución: Se colocó la condición de interrupción del ciclo antes de agregar el elemento a la pila.

Ejercicio 8: 1019. Operaciones con una pila

1. DESCRIPCIÓN BREVE DEL PROBLEMA

El programa recibe una lista de números enteros. Si el tamaño actual de la pila es menor al número leído a, se ejecuta la operación mete(a). En caso contrario, se ejecuta saca(a) eliminando a elementos. Al final se muestra la cantidad restante y su contenido en orden de desapilado.

2. EXPLICACIÓN DEL ALGORITMO

Se construye una clase modular PilaEspecial que envuelve la estructura de pila. Mediante una condición sobre el tamaño de la pila (pila.size() < a), se decide si insertar o realizar un bucle de extracción de a elementos. Finalmente, se invierte la pila a una auxiliar para imprimir los elementos en el orden exacto en que saldrían.

3. PRUEBAS REALIZADAS Y VEREDICTO (OMEGAUP)

4. ENLACE AL CÓDIGO EN GITHUB

Repositorio: https://github.com/ANGEL7231/angel-nolasco-pilas-java/tree/main/ejercicio4_operaciones

5. CONCLUSIONES Y DIFICULTADES

Dificultad: Garantizar que las extracciones masivas no causen una excepción por pila vacía (Underflow).
Solución: Se añadió una comprobación !pila.isEmpty() dentro del bucle de extracción de saca(a).

Ejercicio 10: 3878. Paréntesis Regulares

1. DESCRIPCIÓN BREVE DEL PROBLEMA

Dada una secuencia de caracteres formada por '(' y ')', se busca determinar la longitud de la subcadena regular bien balanceada más larga, junto con el número total de subcadenas distintas que alcanzan dicha longitud máxima.

2. EXPLICACIÓN DEL ALGORITMO

Se almacena en la pila los índices de los caracteres. Se coloca un centinela `-1` al inicio. Al encontrar ')', se extrae el tope y si la pila no queda vacía, la longitud actual se calcula como i - pila.peek(). Se compara este valor con el máximo global para actualizar el récord o incrementar la cuenta de ocurrencias.

3. PRUEBAS REALIZADAS Y VEREDICTO (OMEGAUP) - EJERCICIO 10

Fecha y hora	Lenguaje	Porcentaje	Ejecución	Salida	Memoria	Tiempo	Acciones
2026-11-03 22:48	java	86.67%	Interrumpida	Interrumpida	Límite excedido	4.84 s	

4. ENLACE AL CÓDIGO EN GITHUB

Repositorio: https://github.com/ANGEL7231/angel-nolasco-pilas-java/tree/main/ejercicio5_parentesis

5. CONCLUSIONES Y DIFICULTADES

Dificultad: Calcular la longitud exacta de subcadenas contiguas válidas con complejidad O(N) sin caer en TLE (Time Limit Exceeded).
Solución: El uso de una pila guardando índices de posición permitió obtener la longitud contigua en tiempo constante durante una sola pasada.

Ejercicio 12: 8672. Sumando con pilas

1. DESCRIPCIÓN BREVE DEL PROBLEMA

Se implementa una pila interactiva capaz de procesar 3 comandos: AGREGA v (inserta un valor), CONSUME (retira los 2 elementos superiores, los suma e inserta el resultado) e IMPRIME (muestra el valor del tope actual sin desapilar).

2. EXPLICACIÓN DEL ALGORITMO

Se crea la clase PilaAritmetica con métodos dedicados a cada una de las 3 operaciones. Mediante la lectura de la cadena de comando, se invoca a agrega(v) (hace push), consume() (desapila dos enteros con pop(), calcula a + b y efectúa push de la suma) o imprime() (consulta con peek()).

3. PRUEBAS REALIZADAS Y VEREDICTO (OMEGAUP)

Fecha y hora	Lenguaje	Porcentaje	Ejecución	Salida	Memoria	Tiempo	Acciones
2026-11-03 22:49	java	100.00%	Terminada	Correcta	19.53 MB	0.71 s	

4. ENLACE AL CÓDIGO EN GITHUB

Repositorio: https://github.com/ANGEL7231/angel-nolasco-pilas-java/tree/main/ejercicio6_sumando

5. CONCLUSIONES Y DIFICULTADES

Dificultad: Diferenciar entre la consulta del tope (`peek`) y la extracción permanente (`pop`) requerida para la suma.
Solución: Se definió claramente la responsabilidad del método imprime() para no alterar el contenido de la pila durante la lectura.